

Parallel and HPC HW8 Report

Link to GITHUB repository:

<https://github.com/DMelkonyan111/Parallel-and-HPC-Homeworks>

Exercise 1

The main function generates a DNA sequence of length $N = 10,000,000$ and populates it with random nucleotides {A, C, G, T}. The host prints the first 50 characters to establish a baseline. The data is then copied from the host to four distinct device arrays to test different execution configurations. In the kernel `cuda_func`, each thread calculates its global thread ID (`gid`) and performs a bounds check. If within bounds, the thread reads one character and replaces it with its complement directly in global memory. The program executes this kernel with block sizes of 64, 128, 256, and 512. After synchronization, the results are copied back to the host, and the execution time is measured using `clock_t` to compare performance across different thread configurations.



```
[60] ✓ Os | ./Ex1
... The initial DNA sequence
T, A, A, T, T, A, A, G, A, G, G, G, A, C, T, A, C, G, T, T, G, A, C, C, G, A, A, G, C, T, G, A, T, G, T, G, G, T, A, G, G, A, G, A, A, T, C, G, G,
-----
Results for execution with 64 threads in a block
A, T, T, A, A, T, T, C, T, C, C, C, T, G, A, T, G, C, A, A, C, T, G, G, C, T, T, C, G, A, C, T, A, C, A, C, C, A, T, C, C, C, T, C, T, T, A, G, C, C,
Number of threads executed: 10000000
Execution time 422.000000 ms
-----
Results for execution with 128 threads in a block
A, T, T, A, A, T, T, C, T, C, C, C, T, G, A, T, G, C, A, A, C, T, G, G, C, T, T, C, G, A, C, T, A, C, A, C, C, A, T, C, C, C, T, C, T, T, A, G, C, C,
Number of threads executed: 10000000
Execution time 229.000000 ms
-----
Results for execution with 256 threads in a block
A, T, T, A, A, T, T, C, T, C, C, C, T, G, A, T, G, C, A, A, C, T, G, G, C, T, T, C, G, A, C, T, A, C, A, C, C, A, T, C, C, C, T, C, T, T, A, G, C, C,
Number of threads executed: 10000128
Execution time 227.000000 ms
-----
Results for execution with 512 threads in a block
A, T, T, A, A, T, T, C, T, C, C, C, T, G, A, T, G, C, A, A, C, T, G, G, C, T, T, C, G, A, C, T, A, C, A, C, C, A, T, C, C, C, T, C, T, T, A, G, C, C,
Number of threads executed: 10000384
Execution time 231.000000 ms
```

Exercise 2

The program initializes a DNA sequence of $N = 20,000,000$. Unlike the first task, this implementation requires global counters for each base. An array of four integers is allocated on both the host and the device. The `cuda_func` kernel uses each thread's `gid` to read a nucleotide and then utilizes `atomicAdd` to increment the corresponding index in the device counter array. The use of `atomicAdd` is essential here to prevent data races, as multiple threads frequently attempt to update the same counter simultaneously. The main function launches the kernel four times with varying block sizes (64 to 512). The results show that while the nucleotide counts remain consistent and accurate, the execution time is heavily influenced by "atomic contention," as 20 million threads compete for only four memory locations.

```
[69] ✓ Os !./Ex2
... Results for execution with 64 threads in a block
A: 5002587, C: 5002840, G: 4998762, T: 4995811
In total: 20000000
Number of blocks : 31250, Number of threads in each block: 64
Execution time 3870.000000 ms
-----
Results for execution with 128 threads in a block
A: 5002587, C: 5002840, G: 4998762, T: 4995811
In total: 20000000
Number of blocks : 15625, Number of threads in each block: 128
Execution time 3683.000000 ms
-----
Results for execution with 256 threads in a block
A: 5002587, C: 5002840, G: 4998762, T: 4995811
In total: 20000000
Number of blocks : 7812, Number of threads in each block: 256
Execution time 3679.000000 ms
-----
Results for execution with 512 threads in a block
A: 5002587, C: 5002840, G: 4998762, T: 4995811
In total: 20000000
Number of blocks : 3906, Number of threads in each block: 512
Execution time 3678.000000 ms
-----
```

Exercise 3

This program generates a sequence of $N = 24,000,000$ nucleotides, a size specifically chosen to be divisible by 3. The data is moved to the GPU, and the kernel is launched with a standard block size of 256 threads. Within the kernel, each thread identifies its place in the biological hierarchy by calculating a `codon_id` (determined by `gid / 3`). Simultaneously, the thread identifies its place in the hardware hierarchy by calculating its `warp_id` (determined by `threadIdx.x / 32`) and its `lane_id` (the specific thread position within that warp).

```
[78] ✓ 1s !./Ex3
... Total number of codons: 8000000
Total number of threads: 24000000
Total number of codons: 750000
```